File Edit Selection View Go Run Terminal Help pro2_tema1.cpp - Visual Studio Code

EXPLORER

- pro2_tema1.cpp
- rellotge.hh
- rellotge.cc
- main.cc

pro2_tema1.cpp

```cpp
// Curs: Programació II · UPC
// Grau d'Enginyeria en Informàtica

#include <iostream>

class Tema1_Classes_i_OO {
public:
    void presentacio() {
        string titol = "Classes i Orientació a Objectes";
        string contingut[] = {
            "TADs i Mètodes",
            "Membres Privats",
            "Constructors"
        };
    }
};
```

main* 0 0 0 Ln 14, Col 6 UTF-8 C++ Prettier

const string TEMARI_PRO2[] = { ... };

// Estructures de Dades i Orientació a Objectes

ID	CONTINGUT DEL TEMA	TIPUS
01	**Classes i Orientació a Objectes (C++)**	TEORIA LAB
02	Piles (Stack) i Cues (Queue)	TEORIA LAB
03	Llistes (List) i Iteradors	TEORIA LAB
04	Recursivitat i Arbres Binaris	TEORIA LAB
05	Cues de Prioritat i Arbres Generals	TEORIA
06	BSTs i Diccionaris (Maps)	TEORIA
07	Problemes d'Eficiència	TEORIA
08-12	Punters i Implementació d'Estructures	TEORIA

```
ubuntu@pro2:~$ ./iniciar_tema_1.sh
```

struct TAD_Definition { ... };

// Tipus Abstracte de Dades: Tupla + Operacions

```
/**
 * Un TAD és el resultat d'agrupar:
 * 1. Una tupla (struct) amb dades.
 * 2. Els mètodes per manipular-la.
 */

const Objectiu = "Garantir la consistència";
```

> Exemple: **Rellotge**
> Agrupem les hores, minuts i segons amb les funcions que els modifiquen.

TAD Rellotge

DADES (struct)

```
int hores;
int minuts;
int segons;
```

OPERACIONS (mètodes)

```
bool llegeix();
void escriu()const;
void avansa(ints);
```

void funcions_a_metodes()

// Refactoring: Struct → Class

funcions.cpp — Deleted

```cpp
bool rellotge_llegeix(Rellotge& r);
void rellotge_escriu(const Rellotge& r);
void rellotge_avansa(Rellotge& r, int s);

// Ús:
rellotge_llegeix(r);
rellotge_avansa(r, 10);
```

metodes.cpp — + Added

```cpp
bool llegeix();
void escriu() const;
void avansa(int s);

// Ús:
r.llegeix();
r.avansa(10);
```

COMMIT CHANGES: TRANSFORMATION TO METHODS

- Les funcions es declaren **dins** de la tupla (només la capçalera).
- S'elimina el prefix rellotge_ dels noms.
- Desapareix el paràmetre explícit Rellotge& r (passa a ser implícit).
- La implementació usa el prefix Rellotge:: per indicar pertinença.

// El paràmetre implícit i const

/* Tot mètode opera sobre una instància concreta */

CODI CLIENT (MAIN.CC)

```
Rellotge r;
r.llegeix();

// 'r' és el paràmetre implícit
r.avansa(10);

// 'r' no es modifica aquí
r.escriu();
```

```
void escriu() const;
```

🔒 Promesa de **NO modificar** el paràmetre implícit. Equivalent a rebre const Rellotge&.

VISIÓ INTERNA (DEBUGGER)

Stack Frame: Rellotge::avansa

this	0x7ff... (ptr)	Rellotge*
↳ hores	12	int
↳ minuts	30	int
↳ segons	00	int
s	10	int

Accés directe als camps:

```
segons += s;
// Equival a:
this→segons += s;
```

Separació en fitxers

// Organització modular: Declaració vs Implementació vs Ús

rellotge.hh

```cpp
// Declaració del TAD
#ifndef RELLOTGE_HH
#define RELLOTGE_HH

struct Rellotge {
  int hores, minuts, segons;

  // Només capçaleres
  bool llegeix();
  void escriu() const;
  void avansa(int s);
};

#endif
```

rellotge.cc

```cpp
// Implementació
#include "rellotge.hh"
#include <iostream>
using namespace std;

// Prefix Rellotge::
bool Rellotge::llegeix() {
  // ... codi ...
}

void Rellotge::escriu() const {
  // ... codi ...
}

void Rellotge::avansa(int s) {
  // ... codi ...
}
```

main.cc

```cpp
// Programa principal
#include "rellotge.hh"
#include <iostream>

int main() {
  Rellotge r;
  if (r.llegeix()) {
    r.avansa(10);
    r.escriu();
  }
}
```

Protecció de doble include: Les directives #ifndef, #define i #endif al fitxer .hh eviten errors de redefinició si la capçalera s'inclou múltiples vegades.

class Rellotge { private: ... public: ... };

// Encapsulació: Control d'accés per garantir la consistència

rellotge.hh

```
class Rellotge {
private: // Només intern
  int hores, minuts, segons;

public: // Accés universal
  bool llegeix();
  void escriu() const;
  void avansa(int s);
};
```

Per defecte, en una class tot és private.

ACCÉS PÚBLIC (Permès)

```
r.llegeix();
r.avansa(10);
```

ACCÉS PRIVAT (Prohibit)

```
r.hores=25;
r.minuts=1007;
```

```
main.cc:15:5: error: 'int Rellotge::hores' is private within this context
 15 | r.hores = 25;
 | ^
```

El compilador protegeix la integritat del TAD impedint modificacions arbitràries.

Constructors

// Mètode especial per inicialitzar objectes de forma consistent

- ✓ El seu nom **coincideix** amb el nom de la classe.
- ✓ **No té tipus de retorn** (ni tan sols void).
- ✓ Es crida **automàticament** quan es declara una variable.

AMB PARÀMETRES

DECLARACIÓ (.HH)

```
Rellotge(int h, int m, int s);
```

ÚS (MAIN.CC)

```
Rellotge r1(10, 30, 0);
// Inicialitza a 10:30:00
```

PER DEFECTE

DECLARACIÓ (.HH)

```
Rellotge();
```

ÚS (MAIN.CC)

```
Rellotge r3;
// Inicialitza a 00:00:00
```

DE CÒPIA

DECLARACIÓ (.HH)

```
Rellotge(const Rellotge& a);
```

ÚS (MAIN.CC)

```
Rellotge r2(r1);
// r2 és una còpia exacta de r1
```

Constructor de còpia

// Crear un objecte a partir d'un altre existent

rellotge.cc

```
// Declaració (al .hh)
Rellotge(const Rellotge& altre);

// Implementació
Rellotge::Rellotge(const Rellotge& altre) {
  // Copia camp a camp
  hores = altre.hores;
  minuts = altre.minuts;
  segons = altre.segons;
}
```

Ús al main

```
Rellotge r1(10, 30, 0);
Rellotge r2(r1); // ⟵ Invoca còpia
```

Nota: Si no el declarem, C++ en genera un automàticament que copia tots els membres un per un.

Principis de la POO

// Els 3 pilars fonamentals de la Programació Orientada a Objectes

TEMA 1

01 ENCAPSULACIÓ

Agrupar dades i mètodes en una sola entitat (classe).

Ocultar els detalls interns (membres privats) per protegir la integritat.

FUTUR

02 HERÈNCIA

Una classe pot heretar propietats i comportament d'una altra classe base.

FUTUR

03 POLIMORFISME

Un objecte pot comportar-se de maneres diverses segons el context.

TERMINOLOGIA POO (Mapping) bash

```
user@pro2:~$ cat terminologia.map
  Mètodes ⇒ Operacions del TAD (funcions membre)
  Classe  ⇒ Tupla + Privacitat (struct millorat)
  Objectes ⇒ Variables instanciades de la classe
```

Resum visual: Evolució del Rellotge

// De struct simple a classe encapsulada

Struct + Funcions

v1.0

```
struct Rellotge {
  int h, m, s;
};

void avansa(Rellotge& r, int s) {
  r.s += s; // Accés extern
}
```

Dades públiques | Funcions externes

Struct + Mètodes (TAD)

v2.0

```
struct Rellotge {
  int h, m, s;
  void avansa(int s); // Mètode
};

// Ús: r.avansa(10);
```

Paràmetre implícit | Agrupació

Separació en fitxers

v3.0

```
// rellotge.hh
struct Rellotge { ... };

// rellotge.cc
void Rellotge::avansa(int s) { ... }

// main.cc
#include "rellotge.hh"
```

Modularitat | Compilació separada

Class + Encapsulació

v4.0 (FINAL)

```
class Rellotge {
private:
  int h, m, s; // Ocult
public:
  Rellotge(); // Constructor
  void avansa(int s);
};
```

Consistència garantida

Pròxims temes: Estructures de Dades

// Tot es construeix sobre la base de les Classes

Piles i Cues

TEMA 2

Estructures lineals d'accés restringit (LIFO / FIFO).

`push()` `pop()` `top()`

Llistes i Iteradors

TEMA 3

Seqüències d'elements amb inserció/esborrat eficient.

`insert()` `erase()` `begin()`

Arbres Binaris

TEMA 4

Estructures jeràrquiques recursives.

`left()` `right()` `value()`

Cues de Prioritat

TEMA 5

Accés ràpid al mínim/màxim element (Heap).

`push()` `pop()` `top()`

BSTs i Maps

TEMA 6

Cerca eficient i associació clau-valor.

`find()` `insert()` `[]`

Punters

TEMES 8-12

Gestió manual de memòria i implementació interna.

`new` `delete` `*ptr`

Dominar les **Classes** és el fonament imprescindible per a tot el curs.